A Mini Project Report on

"FACE CHECKI-IN"

Submitted to

OSMANIA UNIVERSITY, Hyderabad

in partial fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING in
COMPUTER SCIENCE AND ENGINEERING (AI & ML)

Submitted by

ABDUL HAADI **161021748008**

Under the guidance of

Dr. Mohammed Sanaullah Qaseem
Assistant Professor of CSE

DEPT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

NAWAB SHAH ALAM KHAN COLLEGE OF ENGINEERING & TECHNOLOGY

Malakpet, Hyderabad.

2023-2024.

Nawab Shah Alam Khan
COLLEGE OF ENGINEERING & TECHNOLOGY
(Autonomous)

Approved by AICTE | Affiliated to Osmania University & SBTET | Accredited by NBA (IT, ME), NAAC 'A'

CERTIFICATE

This is to certify that the Mini Project on **"FACE CHECKI-IN"** is being submitted by the following students:

ABDUL HAADI **161021748008**

They have presented the project work during the academic year 2023–2024 in partial fulfilment of the requirements for the award of **BACHELOR OF ENGINEERING** in **DEPARTMENT OF CSE (AI & ML)** from **Osmania University.**

This is a bonafide record work carried out by them under our guidance and supervision. The results of investigation enclosed with this report have been verified and found to be satisfactory.

Internal Supervisor **Project Coordinator & Head of the Dept.**

External Examiner

DECLARATION

We hereby declare that the project work entitled **"FACE CHECK-IN"** submitted to the Department of **Computer Science and Engineering of NAWAB SHAH ALAM KHAN COLLEGE OF ENGINEERING AND TECHNOLOGY**, affiliated to Osmania University, Hyderabad in the partial fulfilment of the requirement for award of the degree in **BACHELOR OF ENGINEERING** in **CSE (AI & ML)** is a bonafide work done by the undersigned.

ABDUL HAADI **161021748008**

ACKNOWLEDGEMENT

The satisfaction that accomplished completion of any task would be incomplete without the mention of people who made it possible and whose encouragement and guidance has been a source of inspiration through the source of the project.

We express our profound sense of gratitude to **Dr. Syed Abdul Sattar, Principal** for his constant support.

We would like to express our sincere thanks to our Project Supervisor, **Dr. Mohammad Sanaullah Qaseem, Professor of CSE,** for sharing his experience and valuable knowledge to enhance our real time learning experience.

We would like to express our sincere thanks to **Dr. Farheen Sultana, HOD CSE (AI & ML) Dept** and our Internal Guide **Dr. Mohammed Sanaullah Qaseem, Assistant Professor,** for their earnest efforts and timely suggestion and that motivated us to come out with satisfactory project work.

We thank everybody who directly or indirectly played a vital role in finishing our project work with less difficulty.

Names	**Roll No's**
ABDUL HAADI	**161021748008**

CONTENTS

S.no.	Description	Pg.no.
1.	INTRODUCTION	1
2.	LITERATURE SURVEY	5
3.	DESIGN AND MODELLING	8
3.1.	Use case Diagram	
3.2.	Class Diagram	
3.3.	Sequence Diagram	
3.	IMPLEMENTATION	11
3.1.	Present work	
4.	USER INTERFACE	14
4.1.	New user page	
4.2.	Attendance page	
4.3.	Faculty page	
5.	SOURCE CODE	18
5.1.	Stream lit web app	
5.2.	Face classifier	
6.	CONCLUSION	25
7.	REFERENCES	26

Abstract

The "Face Check-In" is an innovative web application designed to streamline the attendance process by harnessing the power of face recognition technology. This system leverages classification algorithms and Python libraries, such as face_recognition and OpenCV-python, to accurately identify individuals and record their attendance in a hassle-free manner.

The core of the system is built upon the face_recognition library, enabling the application to detect and recognize faces with exceptional accuracy. Combined with the functionalities of OpenCV-python, which offers prebuilt functions for image processing, the system efficiently processes incoming visual data, ensuring the seamless identification of candidates.

The web app is developed using the Streamlit library, providing a user-friendly interface that facilitates easy access and management of attendance data. Upon login, candidates' faces are captured and cross-referenced against a pre-registered database, associating each individual with their name, roll number, date, and time of check-in.

The key features of the "Face Check-In" application include real-time face detection, recognition, and attendance logging, making it ideal for educational institutions, corporate offices, and other organizations seeking an automated attendance management solution. The system eliminates the need for manual attendance marking, thus saving time and effort for both candidates and administrators.

In conclusion, "Face Check-In" serves as an efficient and accurate face recognition attendance system that significantly enhances the attendance management process. Its seamless integration of classification algorithms and Python libraries with Streamlit's user-friendly interface sets it apart as a cutting-edge tool for modern-day attendance tracking requirements.

Introduction

FaceCheck-In, a cutting-edge Python web application developed using the Streamlit framework, offers a comprehensive solution to attendance management. This innovative tool harnesses the power of facial recognition technology to revolutionize the way educators take attendance and engage with their students.The application comprises two powerful systems, each tailored to cater to the specific needs of administrators and faculty members. The Admin System empowers administrators with comprehensive insights into all uploaded course data, while the Faculty System empowers faculty members to showcase their course achievements and maintain a personal record of their professional growth.

For students, FaceCheck-In ensures a seamless and non-intrusive experience. There's no need for manual roll calls or sign-in sheets. As they enter the classroom, the application's facial recognition technology swiftly recognizes their faces, updating their attendance status in real-time. This process not only saves time but also cultivates a sense of modernity and technological integration within the learning environment.On the other hand, the Faculty System grants faculty members a user-friendly interface to effortlessly upload their course achievements. Faculty members can enter vital information, such as course names, event details, and certificates, fostering a comprehensive overview of their accomplishments. The system's accessibility and transparency empower faculty members to track their progress, celebrate achievements, and set new professional goals.

The application incorporates a secure professor login feature, ensuring that only authorized educators can access attendance records. Professors can conveniently review attendance logs, track trends, and analyze patterns, all within an intuitive and user-friendly interface. This data-driven approach enables educators to make informed decisions to enhance classroom dynamics and engagement.

Objective

The objectives of the FaceCheck-In application are designed to address the challenges of traditional attendance-taking methods in educational settings and to provide educators with a modern, efficient, and accurate solution for managing student attendance. By leveraging cutting-edge technology and a user-centric approach, FaceCheck-In aims to achieve the following goals:

Key Objectives:

Automate attendance management:

Objective: To replace manual attendance-taking methods with an automated system that utilizes facial recognition technology to accurately track student presence in real-time.

Benefit: This automation eliminates the need for time-consuming manual roll calls, reducing administrative burdens on educators and enhancing overall efficiency.

Enhance accuracy and reliability :

Objective: To ensure precise and reliable attendance records by leveraging advanced facial recognition algorithms that can accurately identify and verify students.

Benefit: The application aims to minimize errors, such as misidentification or unauthorized attendance, resulting in more accurate attendance data for educational institutions..

User Friendly Experience:

Objective: To provide educators with an intuitive and user-friendly interface that requires minimal training and technical expertise.

Benefit: A user-centric design ensures that educators can seamlessly navigate the application, making attendance management more accessible and efficient.

Scope

The scope of FaceCheck-In encompasses the development of a sophisticated Python web application using the Streamlit framework, dedicated to transforming attendance management within educational environments. The application's primary focus lies in leveraging advanced facial recognition technology to provide educators with a streamlined and efficient method for recording student attendance. This system will automatically identify and mark the presence of students as they enter the classroom, eliminating the need for traditional manual attendance-taking methods.

The application will also include a secure professor login portal, granting authorized educators access to attendance records and real-time updates. Additionally, the system will facilitate data analysis and reporting, empowering educators to make informed decisions based on attendance trends. With a user-friendly interface and compatibility across various devices, FaceCheck-In aims to enhance classroom engagement, optimize administrative processes, and foster a technologically integrated educational experience.

Literature Survey

Attendance management is a critical aspect of the education system, impacting student engagement, academic success, and administrative efficiency. Traditional methods of manual attendance tracking have proven to be time-consuming and error-prone. In recent years, various technological solutions have emerged to address these challenges and streamline the attendance management process. This literature survey provides an overview of modern approaches, technologies, and tools that have been explored for attendance management in educational settings.

1. "**Advances in Face Detection and Facial Image Analysis**" by Ioannis Pitas

This book delves into the fundamental concepts and advanced techniques of face detection and facial image analysis. It explores various algorithms and methodologies used in facial recognition systems, making it a valuable resource for understanding the technical aspects of facial recognition technology. While not exclusively focused on attendance systems, the knowledge gained from this book can be applied to designing robust facial recognition components within attendance management solutions.

2. "**Facial Analysis from Continuous Video with Applications to Human-Computer Interface**" by Ziyou Xiong, Fernando De la Torre, and Jeffrey F. Cohn

This book focuses on facial analysis techniques and their applications, including human-computer interfaces. It covers the analysis of facial expressions, emotions, and other facial attributes. While its primary emphasis is on broader applications, the insights provided in this book can be useful for designing attendance systems that not only recognize faces but also extract meaningful information from facial expressions and gestures.

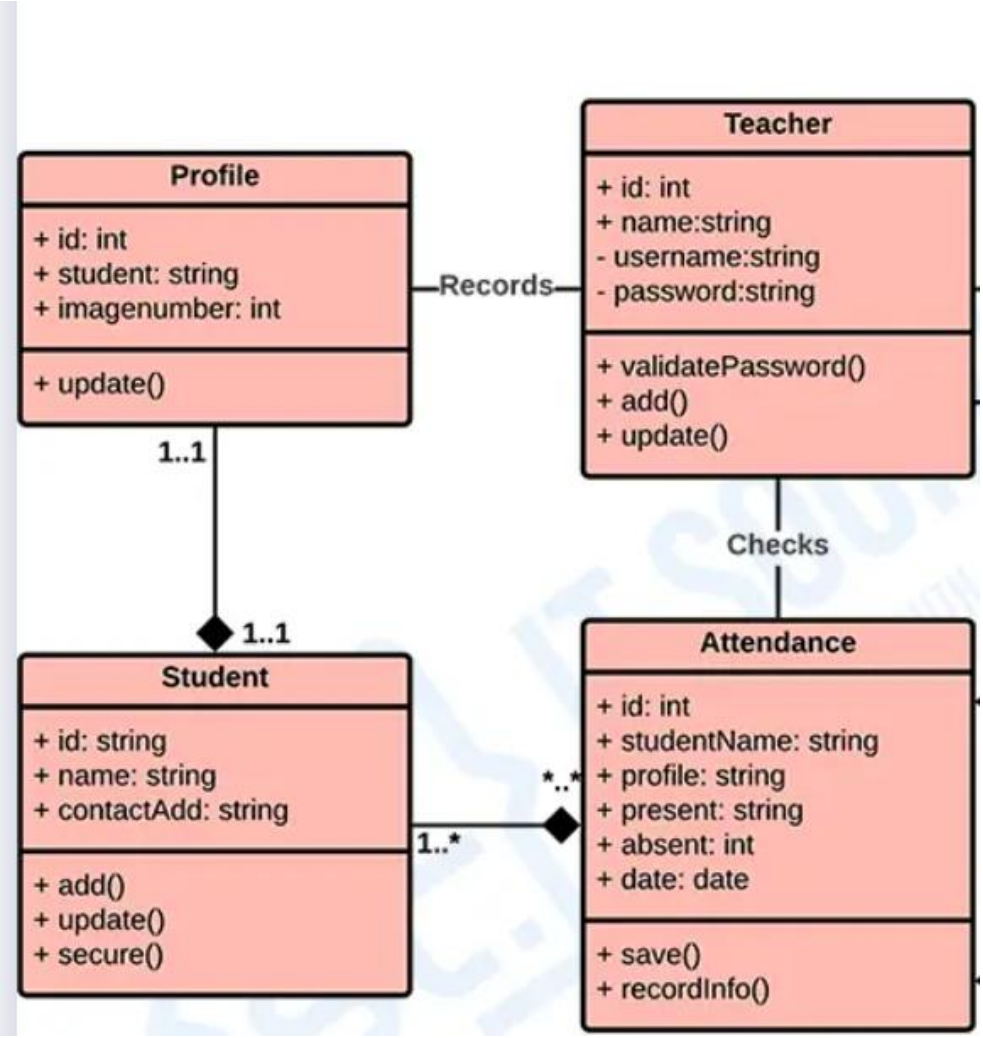
3. "**Biometric Recognition: Challenges and Opportunities**" by Anil K. Jain, Ruud Bolle, and Sharath Pankanti

This comprehensive book discusses various biometric recognition methods, including facial recognition, and their challenges and opportunities. It provides

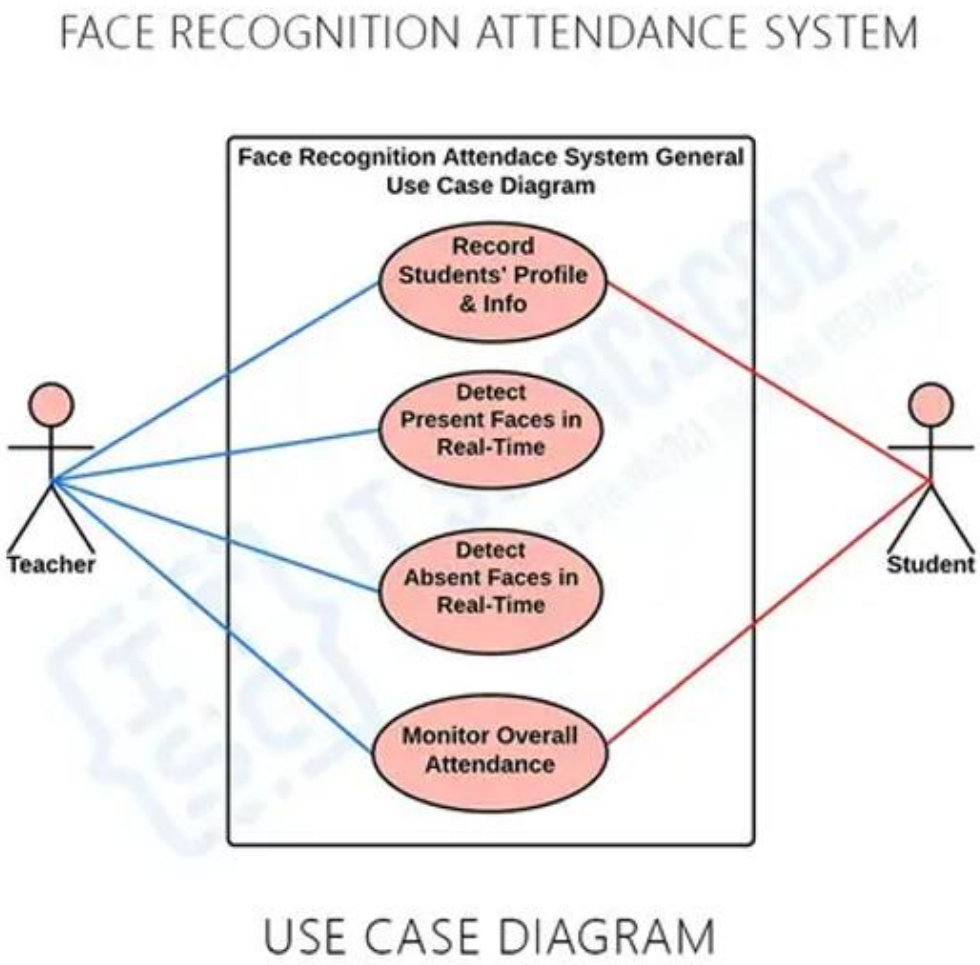
a deeper understanding of the technical and practical aspects of biometric systems.

System Design and Modelling

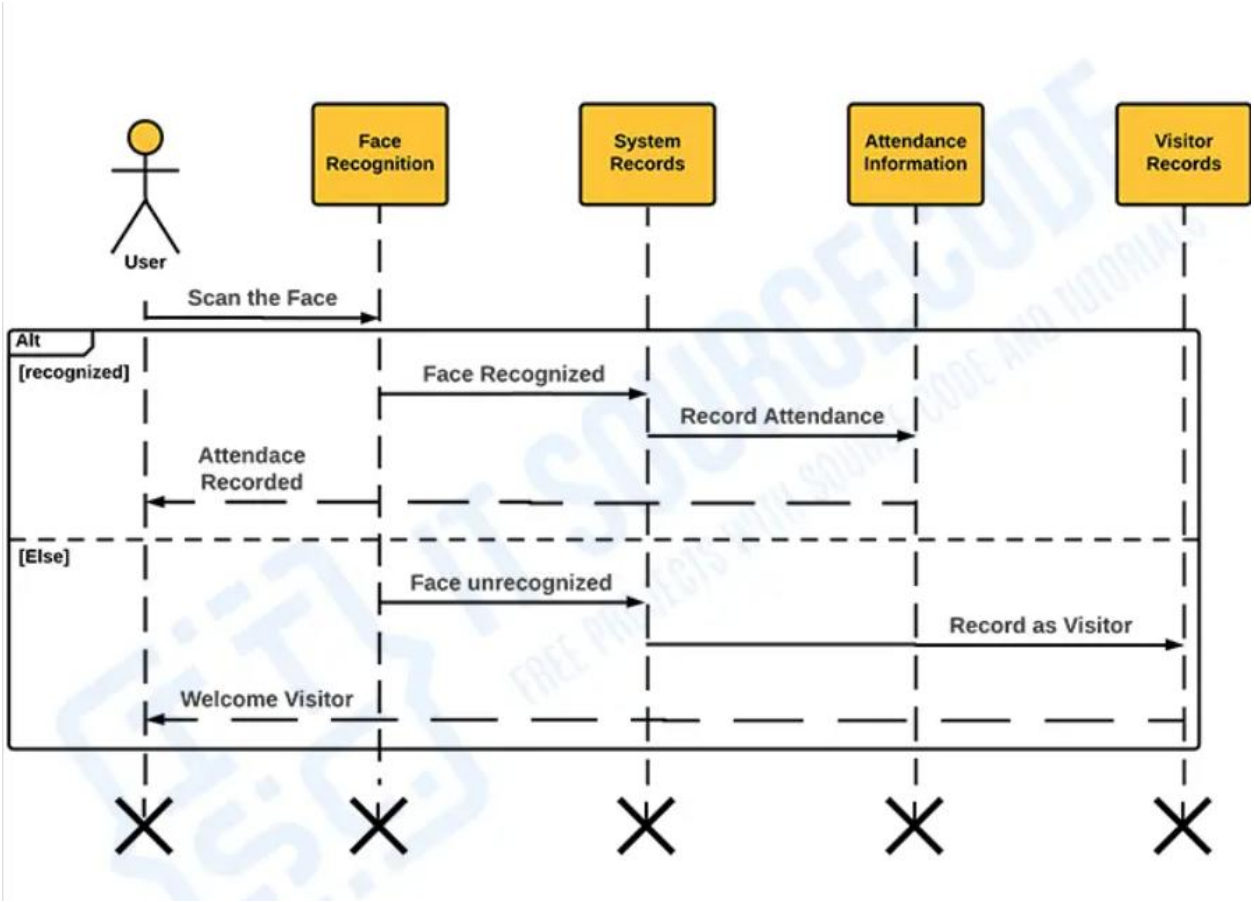
UML Diagrams Class Diagram



Use case diagram



Sequence Diagram



Implementation

Code Editor

1. Visual Studio Code (or any code editor): A code editor used for writing, editing, and managing the website's code.

These tools and technologies collectively enable the implementation of various features on your website, such as user authentication, course registration, data storage, data retrieval, and front-end design. The combination of these technologies ensures the website's functionality, security, and user-friendliness.

Present Work

Step 1: Project Setup and Environment:

Create a new project directory.
Set up a virtual environment to manage dependencies.
Install required libraries, such as OpenCV, face_recognition, Streamlit, and any other necessary packages.

Step 2: Data Collection and Preprocessing

Collect a dataset of images of individuals for attendance.
Organize the dataset into separate folders for each person.
Use image preprocessing techniques like resizing and normalization to enhance accuracy.
Detect and encode facial features using the face_recognition library.

Step 3: Model Training

If needed, train a custom face recognition model using machine learning algorithms like SVM or deep learning frameworks like TensorFlow or PyTorch.
Fine-tune the model using the pre-processed dataset.
Save the trained model for inference.

Step 4: Streamlit App Creation

Create a new Streamlit app Python script.
Import necessary libraries and modules.
Design the user interface (UI) using Streamlit components like st.title(), st.sidebar(), and st.button().
Add an option for users to upload an image for attendance.

Step 5: Face Recognition and Attendance Tracking

Handle image upload using Streamlit's file input component.
Process the uploaded image using OpenCV and face_recognition.
Compare the detected face features with the preprocessed dataset.
If a match is found, mark attendance for the corresponding individual.

Step 6: Display Results

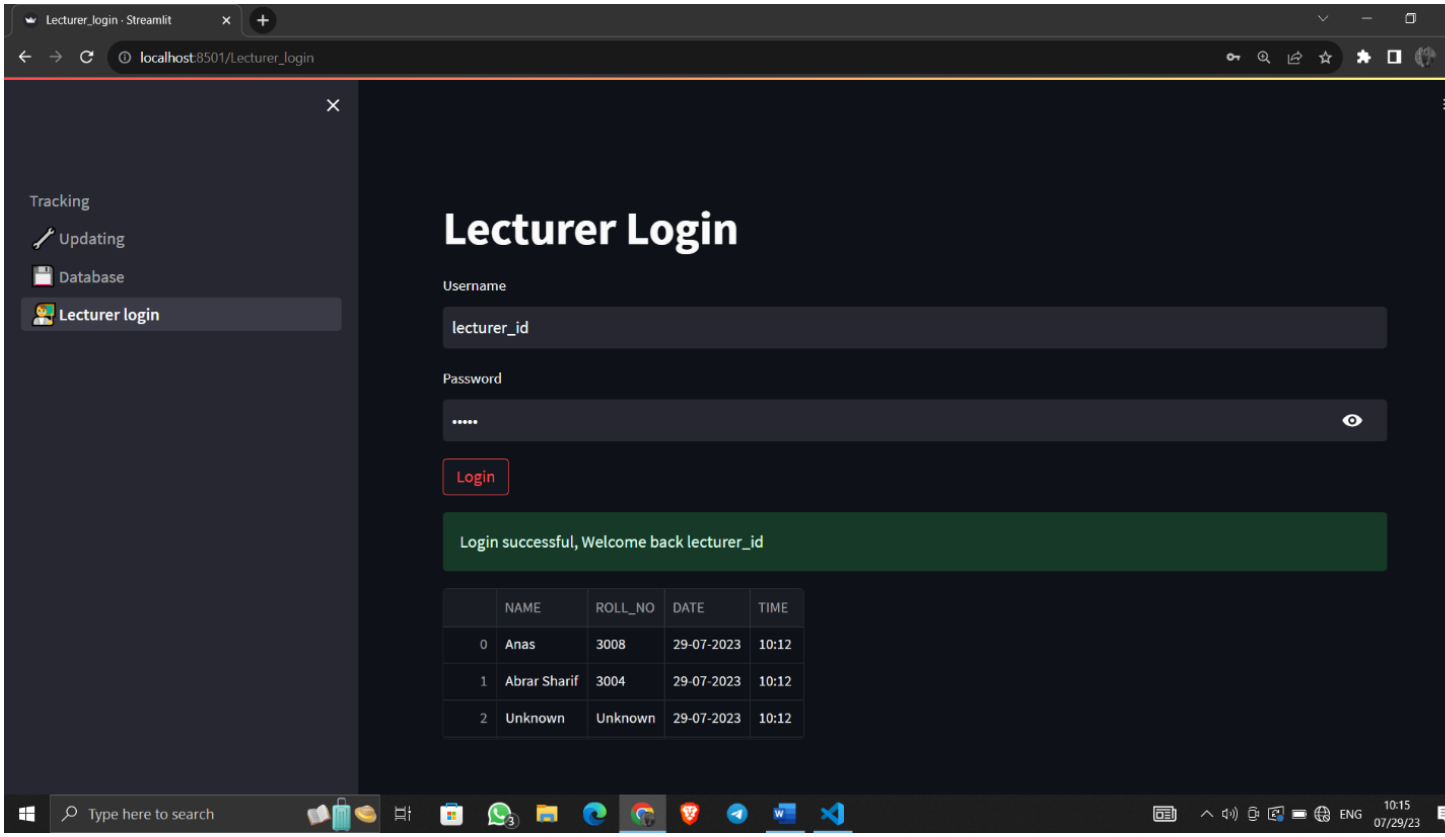
Display the uploaded image with detected faces and marked attendance.
Provide feedback to the user about the recognition results.

User Interface

1. Faculty Login:

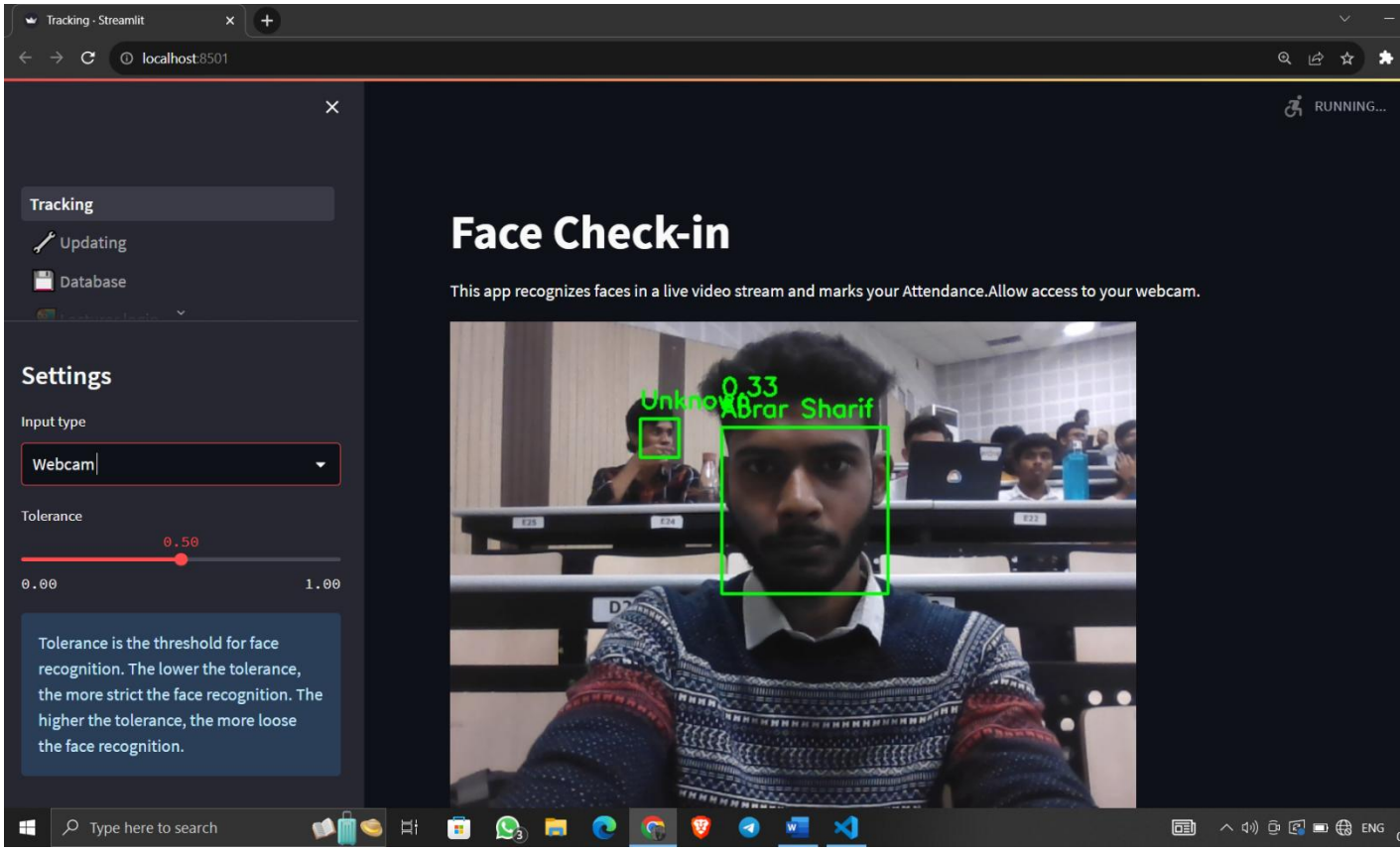
- When a faculty member visits the "Faculty Login" page, they encounter a form where they need to input their username and password.

- After entering the credentials, they click the "Login" button, triggering a POST request to the PHP backend script responsible for handling login authentication.



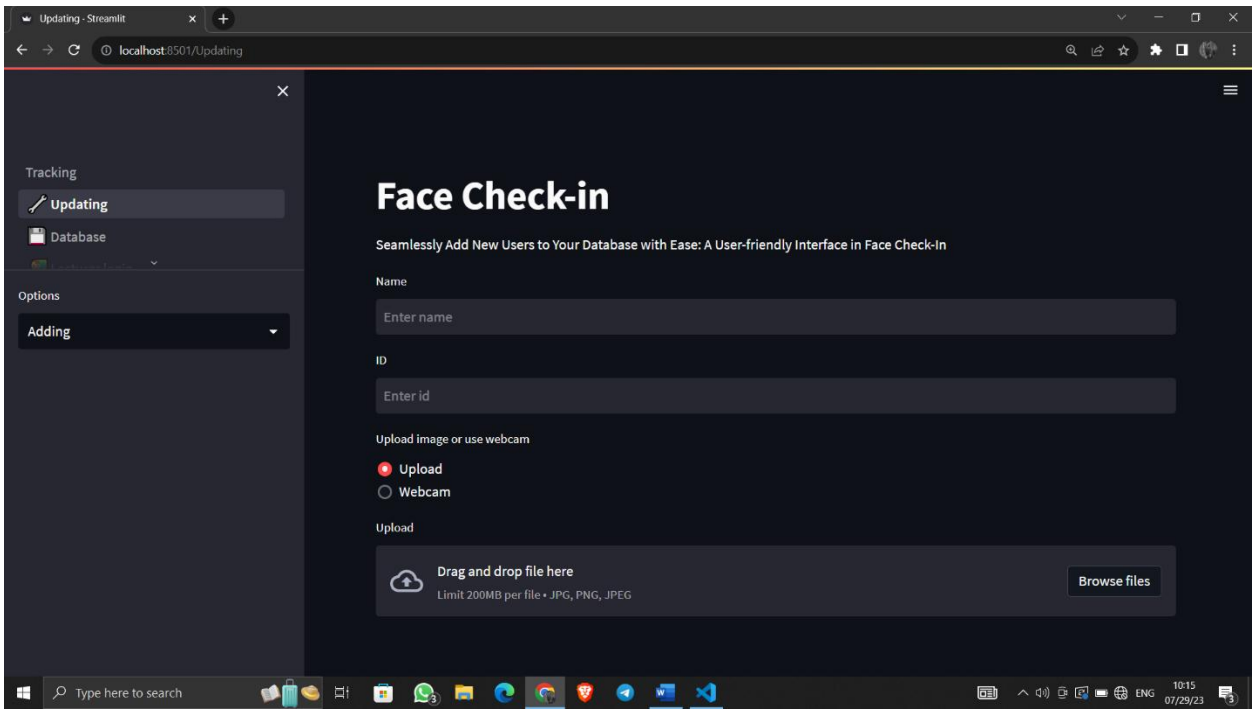
Attendance Tracking:

Face recognition takes place then the name of the person is identified and stored in a csv file.

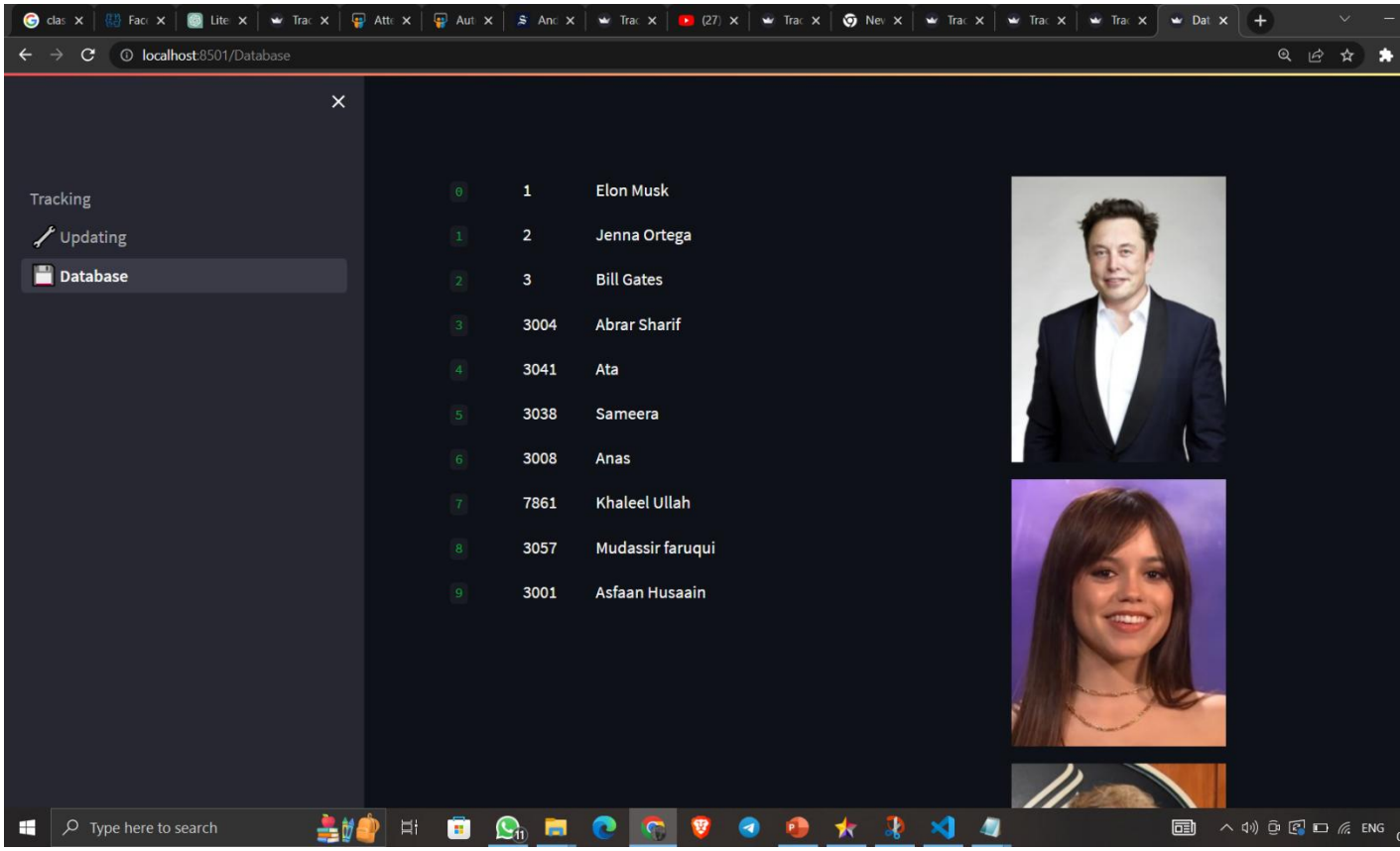


New Student registration:

New students must fill in details in order to get recognized



View Database:



Source Code:

```
import streamlit as st
import cv2
import face_recognition as frg
import yaml
from utils import recognize, build_dataset, mark
import csv
import time
from datetime import datetime
import time as t
import pandas as pd
# Path: code\app.py
st.set_page_config(layout="wide")
#Config
cfg = yaml.load(open('config.yaml','r'),Loader=yaml.FullLoader)
PICTURE_PROMPT = cfg['INFO']['PICTURE_PROMPT']
WEBCAM_PROMPT = cfg['INFO']['WEBCAM_PROMPT']
st.sidebar.title("Settings")
#Create a menu bar
menu = ["Picture","Webcam"]
choice = st.sidebar.selectbox("Input type",menu)
#Put slide to adjust tolerance
TOLERANCE = st.sidebar.slider("Tolerance",0.0,1.0,0.5,0.01)
st.sidebar.info("Tolerance is the threshold for face recognition. The lower the tolerance, the more
strict the face recognition. The higher the tolerance, the more loose the face recognition.")
#Infomation section
st.sidebar.title("Student Information")
name_container = st.sidebar.empty()
id_container = st.sidebar.empty()
name_container.info('Name: Unnknown')
```

```
id_container.success('ID: Unknown')
if choice == "Picture":
    st.title("Face Check-in")
    st.write(PICTURE_PROMPT)
    uploaded_images =
st.file_uploader("Upload",type=['jpg','png','jpeg'],accept_multiple_files=True)
    if len(uploaded_images) != 0:
        #Read uploaded image with face_recognition
        for image in uploaded_images:
            image = frg.load_image_file(image)
            image, name, id = recognize(image,TOLERANCE)
            name_container.info(f"Name: {name}")
            id_container.success(f"ID: {id}")
            st.image(image)
    else:
        st.info("Please upload an image")

elif choice == "Webcam":
    st.title("Face Check-in")
    st.write(WEBCAM_PROMPT)
    #Camera Settings
    cam = cv2.VideoCapture(0)
    cam.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cam.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    FRAME_WINDOW = st.image([])

    while True:
        ret, frame = cam.read()
        if not ret:
            st.error("Failed to capture frame from camera")
            st.info("Please turn off the other app that is using the camera and restart app")
            st.stop()
        image, name, id = recognize(frame,TOLERANCE)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
        #Display name and ID of the person

        name_container.info(f"Name: {name}")
        id_container.success(f"ID: {id}")
        FRAME_WINDOW.image(image)
        mark(name, id)#marks attendance

with st.sidebar.form(key='my_form'):
    st.title("Developer Section")
    submit_button = st.form_submit_button(label='REBUILD DATASET')
    if submit_button:
        with st.spinner("Rebuilding dataset..."):
            build_dataset()
        st.success("Dataset has been reset")
```

Face Classifier and other functions:

```
import face_recognition as frg
import pickle as pkl
import os
import cv2
import numpy as np
import yaml
from collections import defaultdict
import csv
import time
from datetime import datetime
import time as t


COL_NAMES = ['NAME', 'ROLL_NO', 'DATE','TIME']

information = defaultdict(dict)
cfg = yaml.load(open('config.yaml','r'),Loader=yaml.FullLoader)
DATASET_DIR = cfg['PATH']['DATASET_DIR']
PKL_PATH = cfg['PATH']['PKL_PATH']

def get_databse():
    with open(PKL_PATH,'rb') as f:
        database = pkl.load(f)
    return database
def recognize(image,TOLERANCE):
    database = get_databse()
    known_encoding = [database[id]['encoding'] for id in database.keys()]
    name = 'Unknown'
    id = 'Unknown'
    face_locations = frg.face_locations(image)
    face_encodings = frg.face_encodings(image,face_locations)
    for (top,right,bottom,left),face_encoding in zip(face_locations,face_encodings):
```

```
        matches =
frg.compare_faces(known_encoding,face_encoding,tolerance=TOLERANCE)
        distance = frg.face_distance(known_encoding,face_encoding)
        name = 'Unknown'
        id = 'Unknown'
        if True in matches:
            match_index = matches.index(True)
            name = database[match_index]['name']
            id = database[match_index]['id']
            distance = round(distance[match_index],2)
            cv2.putText(image,str(distance),(left,top-
30),cv2.FONT_HERSHEY_SIMPLEX,0.75,(0,255,0),2)
        cv2.rectangle(image,(left,top),(right,bottom),(0,255,0),2)
        cv2.putText(image,name,(left,top-10),cv2.FONT_HERSHEY_SIMPLEX,0.75,(0,255,0),2)
    return image, name, id
def isFaceExists(image):
    face_location = frg.face_locations(image)
    if len(face_location) == 0:
        return False
    return True
def submitNew(name, id, image, old_idx=None):
    database = get_databse()
    #Read image
    if type(image) != np.ndarray:
        image = cv2.imdecode(np.fromstring(image.read(), np.uint8), 1)

    isFaceInPic = isFaceExists(image)
    if not isFaceInPic:
        return -1
    #Encode image
    encoding = frg.face_encodings(image)[0]
    #Append to database
    #check if id already exists
    existing_id = [database[i]['id'] for i in database.keys()]
```

```
    #Update mode
    if old_idx is not None:
        new_idx = old_idx
    #Add mode
    else:
        if id in existing_id:
            return 0
        new_idx = len(database)
    image = cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
    database[new_idx] = {'image':image,
                         'id': id,
                         'name':name,
                         'encoding':encoding}
    with open(PKL_PATH,'wb') as f:
        pkl.dump(database,f)
    return True
def get_info_from_id(id):
    database = get_databse()
    for idx, person in database.items():
        if person['id'] == id:
            name = person['name']
            image = person['image']
            return name, image, idx
    return None, None, None
def deleteOne(id):
    database = get_databse()
    id = str(id)
    for key, person in database.items():
        if person['id'] == id:
            del database[key]
            break
    with open(PKL_PATH,'wb') as f:
        pkl.dump(database,f)
    return True
```

```
def build_dataset():
    counter = 0
    for image in os.listdir(DATASET_DIR):
        image_path = os.path.join(DATASET_DIR,image)
        image_name = image.split('.')[0]
        parsed_name = image_name.split('_')
        person_id = parsed_name[0]
        person_name = ' '.join(parsed_name[1:])
        if not image_path.endswith('.jpg'):
            continue
        image = frg.load_image_file(image_path)
        information[counter]['image'] = image
        information[counter]['id'] = person_id
        information[counter]['name'] = person_name
        information[counter]['encoding'] = frg.face_encodings(image)[0]
        counter += 1

    with open(os.path.join(DATASET_DIR,'database.pkl'),'wb') as f:
        pkl.dump(information,f)

def mark(name, id):
    # This function keeps track of faces detected and stores name, roll no, date and time in
att_csv file
    ts = time.time()
    date=datetime.fromtimestamp(ts).strftime("%d-%m-%Y")
    timestamp=datetime.fromtimestamp(ts).strftime("%H:%M")
    exist=os.path.isfile("Attendance/att_" + date + ".csv")

    ate = [str(name), str(id),str(date),str(timestamp)]

    #if k==ord('k'):
    if exist:
            with open("Attendance/att_" + date + ".csv", "+a") as csvfile:
              writer= csv.writer(csvfile)
```

```
            writer.writerow(ate)
    csvfile.close()
        else:
            with open("Attendance/att_" + date + ".csv", "+a") as csvfile:
                writer= csv.writer(csvfile)
                writer.writerow(COL_NAMES)
                writer.writerow(ate)
            csvfile.close()
if __name__ == "__main__":
        deleteOne(4)
```

Conclusion

In conclusion, the FaceCheck-In application stands as a testament to the transformative potential of technology in redefining attendance management within educational domains. By seamlessly integrating advanced facial recognition technology into the conventional practice of attendance tracking.

FaceCheck-In offers educators a modern, efficient, and accurate solution. The real-time updates, automated attendance marking, and data-driven insights empower educators to not only streamline administrative tasks but also cultivate a more engaging and interactive classroom atmosphere. With a secure professor login, user-friendly interface, and compatibility across devices.

FaceCheck-In caters to the diverse needs of modern education, whether in traditional classrooms or hybrid learning environments. As education continues to evolve in the digital age, FaceCheck-In emerges as an innovative tool that bridges the gap between technological innovation and effective pedagogy, ultimately enhancing the educational experience for both educators and students alike.

References

1) https://github.com/ageitgey/face_recognition
2) https://www.youtube.com/watch?v=BYCKvM8eZGA&t=2782s
3) https://www.researchgate.net/publication/341876647_Face_Recognition_based_Attendance_Management_System
4) https://ieeexplore.ieee.org/document/9215441